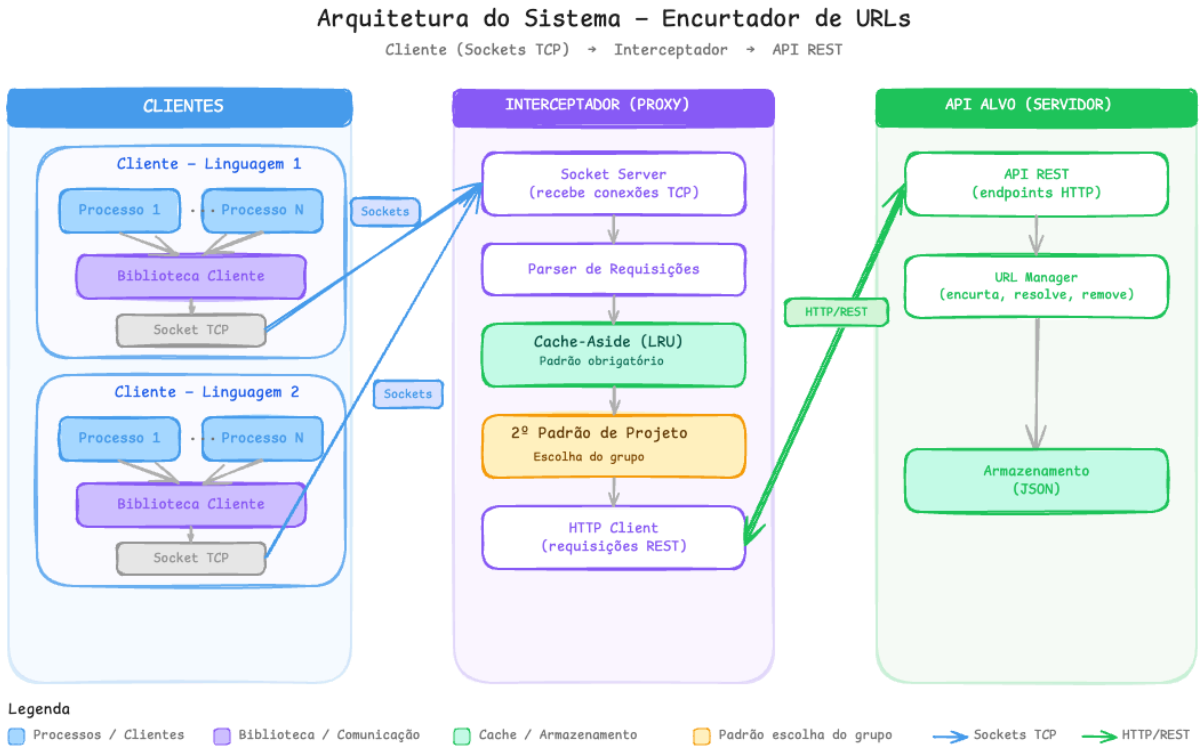
Universidade Federal de Santa Catarina – UFSC
Departamento em Informática e Estatística
INE 5418 – Computação Distribuída
Semestre 2026/1

Definição do Trabalho 1: Encurtador de URLs Distribuído

Serviços de encurtamento de URLs permitem transformar URLs longas em códigos curtos e fáceis de compartilhar, além de possibilitar o rastreamento de acessos. Neste trabalho, deve ser implementado um sistema de encurtamento de URLs simplificado, com foco na comunicação distribuída entre componentes.

A arquitetura do sistema (ilustrada na figura abaixo) é composta por três elementos principais: **clientes, um interceptador (proxy) e um servidor REST (API alvo)**. Os clientes se comunicam com o interceptador via **sockets** (usar API Berkley sockets) e o interceptador repassa as requisições ao servidor via **API REST** (HTTP). O interceptador atua como um ***middleware***, podendo aplicar lógicas como cache, controle de tráfego ou tratamento de falhas antes de encaminhar as requisições ao servidor.



API Alvo (Servidor REST)

O servidor expõe uma API REST para gerenciamento de URLs encurtadas. É uma aplicação independente que não possui conhecimento do interceptador: recebe requisições HTTP e retorna respostas. O armazenamento dos mapeamentos (código curto -> URL

original) é feito localmente (em memória).

Endpoints da API REST:

Método	Endpoint	Descrição	Corpo / Resposta
POST	/urls	Encurta uma URL	Req: { "url": "https://..." } Res: { "codigo": "abc123", "url_curta": "http://host/r/abc123" }
GET	/urls/{codigo}	Resolve um código curto	Res: { "url_original": "https://..." }
DELETE	/urls/{codigo}	Remove uma URL encurtada	Res: { "removido": true }
GET	/urls	Lista todas as URLs encurtadas	Res: [{ "codigo": "...", "url_original": "...", "acessos": ... }]

Interceptador (Proxy)

O interceptador é o componente central do trabalho. Ele:

- Recebe requisições dos clientes via *sockets* TCP
- Processa as requisições aplicando padrões de projeto escolhidos (cache, circuit breaker, rate limiting, etc.)
- Repassa as requisições ao servidor via HTTP/REST
- Retorna as respostas ao cliente via *sockets*
- O interceptador deve ser transparente para o servidor, o servidor não sabe que existe um *proxy* no meio. Do ponto de vista do cliente, o interceptador é o "servidor".

Cliente

O cliente é um programa que se conecta ao interceptador via *sockets* TCP e envia comandos para encurtar, resolver ou remover URLs. A API disponível ao cliente deve ser oferecida por uma biblioteca, de modo que qualquer programador possa utilizar o serviço importando a biblioteca.

Funções da biblioteca:

- `int encurta(char *url_original, char *url_curta)`: Envia a URL original ao interceptador e recebe o código curto. Retorna código de erro em caso de falha.
- `int resolve(char *codigo_curto, char *url_original)`: Envia o

código curto ao interceptador e recebe a URL original. Retorna código de erro em caso de falha.
- `int remove_url(char *codigo_curto)`: Remove o mapeamento de uma URL encurtada. Retorna código de erro em caso de falha.

Coerência de *cache* no Interceptador

O interceptador mantém um *cache* local dos mapeamentos de URLs resolvidas recentemente (código curto -> URL original). Este *cache* utiliza o padrão **Cache-Aside** e tem como objetivo reduzir requisições ao servidor REST, melhorando o tempo de resposta para os clientes.

No caso de resoluções sucessivas, se o interceptador tiver uma cópia válida do mapeamento em *cache*, a resposta é retornada diretamente ao cliente via *socket*, sem gerar requisição ao servidor REST. Caso contrário (*cache miss*), o interceptador consulta o servidor, armazena a resposta em *cache* e retorna ao cliente.

No caso de remoções, quando um cliente solicita a remoção de uma URL encurtada, o interceptador encaminha a requisição DELETE ao servidor REST e invalida a entrada correspondente no *cache* local, garantindo que consultas futuras não retornem dados obsoletos.

A política de gerenciamento do *cache* (ex: LRU - *Least Recently Used*) e sua capacidade máxima ficam a critério do grupo e devem ser configuráveis. É permitida a utilização de estratégias complementares (*fallback*) como TTL (*Time-to-Live*) para expiração automática de entradas.

Heterogeneidade

O trabalho deve demonstrar o potencial de heterogeneidade. Para isso, além da implementação do cliente em uma determinada linguagem de programação, será necessário disponibilizar a biblioteca cliente para uma segunda linguagem de programação (ex. clientes em C e JavaScript, ou Python e Java, etc.).

Como a comunicação entre cliente e interceptador é feita via *sockets* TCP, qualquer linguagem que suporte *sockets* pode se conectar, basta implementar a biblioteca que traduz as funções da API para o protocolo de mensagens definido.

Configuração

Na inicialização do sistema, é necessário iniciar o servidor REST e o interceptador. Arquivos de configuração podem ser utilizados para manter informações necessárias. Por exemplo, o endereço e a porta do servidor REST, o endereço e a porta do interceptador (para os clientes), e parâmetros do cache podem estar disponíveis no arquivo `config.txt`.

O formato de requisições e respostas por solicitação de arquivo fica a critério do grupo, assim como a estratégia para a implementação de cache e informações adicionais necessárias para a invalidação de *cache*.

Uso de padrões de projeto para computação distribuída

Padrão obrigatório - *Cache-Aside*: O interceptador deve manter um *cache* local

das resoluções de URLs. Quando um cliente solicita a resolução de um código curto, o interceptador verifica se a resposta está em *cache* antes de encaminhar a requisição ao servidor REST. Se estiver em *cache* (*cache hit*), retorna diretamente ao cliente sem contatar o servidor. Se não estiver (*cache miss*), consulta o servidor, armazena a resposta em *cache* e retorna ao cliente. Quando uma URL é removida, a entrada correspondente deve ser invalidada no *cache*.

Segundo padrão obrigatório, a critério do grupo: Adicionalmente, o grupo deve adotar um segundo padrão de projeto visando melhorar atributos como disponibilidade, desempenho ou tolerância a falhas. As referências a seguir podem ser úteis para a escolha: https://learn.microsoft.com/pt-br/azure/architecture/patterns/; https://www.enterpriseintegrationpatterns.com/

Execução

Para realizar testes sobre o sistema implementado, cada grupo deve implementar clientes de exemplo.

- Deve ser possível ilustrar o uso de clientes em diferentes linguagens de programação;
- Deve ser possível observar o uso do *cache* no interceptador, com resoluções sucessivas de um mesmo código curto;
- Deve ser possível observar a invalidação do cache quando uma URL é removida;
- Deve ser possível observar o comportamento do segundo padrão de projeto escolhido (ex: ao derrubar o servidor REST, demonstrar o *Circuit Breaker*; ao enviar muitas requisições simultâneas, demonstrar o *Throttling*).

Entrega

O trabalho consiste em:

1. Implementar o servidor REST, o interceptador e a biblioteca cliente em ao menos 2 linguagens de programação diferentes, além de programas clientes de exemplo que validem as funcionalidades oferecidas.
2. Breve relatório que indique as principais decisões e estratégias de implementação utilizadas, instruções sobre como compilar e executar o código produzido, além da justificativa da escolha do padrão de projeto e de exemplos de saídas de execução com diferentes parametrizações. Ao final, discuta quais conclusões você observa, indicando limitações observadas na implementação atual.

O trabalho pode ser realizado em **grupos de até 3 participantes**. O trabalho será apresentado em laboratório.

O código-fonte e relatório devem ser enviados pelo Moodle para análise e avaliação.

Os nomes dos participantes do grupo devem constar no relatório entregue no Moodle. Participantes com nomes não referenciados não serão considerados como membros do grupo.